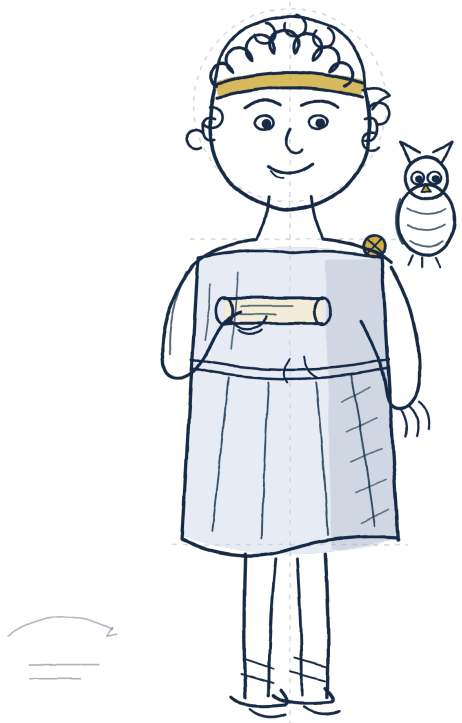
TMach 開発者ガイド

Racket を軸にした、自分たちのインフラで動かすチーム向け AI プラットフォームの小さな本 — 本日本語版は全章訳出済みですが、機械翻訳であり、ネイティブによる校閲は未了です。疑わしい箇所は英語版が正文です (machine-translated; pending native review — the English edition is authoritative)



IoTone, Inc.

2026 年 9 月 — バージョン 0.3.0

テレマコス、コンセプトスケッチ：巻物を持つ若きイタケ人と、肩に乗るメントールのフクロウ。



この本の読み方

TMach（本プロジェクトは Telemachus として始まり、長い名前はリポジトリと環境変数に残っている）は、チームの AI ツールとアプリケーションを自分たちのインフラで動かすためのプラットフォームである。この本は、リポジトリをクローンし、変更を加える前にその構造を理解したい開発者のためのものである。短いのは意図的である：`docs/reference/` にある生成されたリファレンスが、すべてのツール、ワークフロー、権限、HTTP ルートを唯一の出典から記録しており、`docs/design/` にある設計ドキュメントが、各サブシステムがなぜその形をしているのかを記録している。この本はその二つをつなぐ組織である — 繰り返し現れる考え方、学ぶのに高くついた決定、そしてコードが意味を持ち始める前に必要となる地図である。

各章の終わりには、開いてみるべきファイルの短い一覧がある。最初の 4 章は順番に読んでほしい。それ以降は、自分の課題が連れて行く先へ進めばよい。



Contents

1 TMach とは何か **5**

1.1 一段落で言うと 5

1.2 原則 5

1.3 どこに何があるか 6

2 動かしてみる **7**

2.1 ツールチェーン 7

2.2 環境変数 7

2.3 最初の起動 8

2.4 開いてみるファイル 8

3 コードの地図 **9**

3.1 層 9

3.2 四つの小さなライブラリ 9

3.3 プリンシパル一つ、検査一つ 10

3.4 開いてみるファイル 10

4 認可のモデル **11**

4.1 権限とロール 11

4.2 `can?` の順序 11

4.3 権限のまとまり 12

4.4 マルチテナンシー 12

4.5 開いてみるファイル 12

5 データとマイグレーション **13**

5.1 開いてみるファイル 13

6 ドキュメント保管庫 **14**

6.1 オブジェクト、バージョン、ブロブ 14

6.2 S3 エンドポイント 14

6.3 共有と来歴 14

6.4 開いてみるファイル 15

7 ツールとエージェント **16**

7.1 ツールを宣言する 16

7.2 レジストリ 16

7.3 モデルとの継ぎ目 17

7.4 メッセージが運べるもの 17

7.5 開いてみるファイル 17

8 ワークフロー **18**

8.1 仕様が契約である 18

8.2 実行はリデューサである 18

8.3 開いてみるファイル 19

9 ドキュメントパイプライン **20**

9.1 最初の利用者の道筋 20

9.2 印をつけるのではなく拒否する 20

9.3 テンプレート 20

9.4 トリガ 21

9.5 開いてみるファイル 21

10 翻訳管理 **22**

10.1 三つの住所 22

10.2 翻訳マネージャ 22

10.3 開いてみるファイル 22

11 知識グラフ **23**

11.1 開いてみるファイル 23

12 ルート表と生成されるリファレンス **24**

12.1 制定ではなく宣言 24

12.2 生成され、ゲートされる 24

13 ジョブの待ち行列 **25**

13.1 ジョブとは何か 25

13.2 一つを確保する 25
13.3 ワーカーが死ぬとき 26
13.4 仕事の側へやって来る実行基盤 26
13.5 どのモデルが何を走らせるか 26
13.6 開いてみるファイル 27

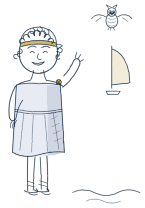
14 秘密と玄関 **28**
14.1 そもそも誰が呼んでよいのか 28
14.2 何がハッシュ化され、何ができないのか 28
14.3 残りを封じる 29
14.4 二要素認証と、戻る道 29
14.5 セッションはどれだけ続くか 30
14.6 コンソールの方針 30
14.7 開いてみるファイル 31

15 TMach を拡張する **32**
15.1 四つの継ぎ目と、一つの接頭辞 32
15.2 プラグインはどこで見つかるのか、そして見つからないとどうなるのか . . . 32
15.3 自分の画面 33
15.4 顧客の色をまとう 33
15.5 使えないもの 34
15.6 開いてみるファイル 34

16 このプロジェクトはどう試験するのか **35**

17 運用の覚書 **37**

18 貢献する **39**



Chapter 1

TMach とは何か

1.1 一段落で言うと

チームが自分たちのドキュメントをアップロードし、それについてモデルと対話し、その上でワークフローを走らせ、結果を見るべき人にだけ共有する — そのすべてを、チームが管理するマシンの上で行う。プラットフォームの実体は、HTTP API、ブラウザコンソール、S3 互換エンドポイント、ワークフローエンジン、スケジューラ、クォータ台帳、プラグイン機構を備えた一つの Racket プログラムである。基盤より上のあらゆる機能 — 翻訳管理、ドキュメントパイプライン、知識グラフ — は、その基盤自身のプリミティブから組み立てられている。それがアーキテクチャそのものであり、同時にアーキテクチャが機能することの証明でもある。

1.2 原則

これらはスローガンではない。どれも少なくとも一度は設計を変えさせた。

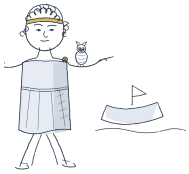
- **クリーンな MIT、純粋なオープンソース**。オープンコアはなく、出し惜しみする上位版もない。以前の Python システムから概念を再利用することはあるが、持ち込まれたのは所有者が書いた Racket と所有者が書いたドキュメントだけである。
- **Racket ファースト**。リファレンス実装は Racket 9.3 CS である。Racket 以外の言語でプラグインを書けることは、明確に要件としていない。データ形式が契約になっている箇所（ワークフロー仕様、ツールスキーマ）があるのは、その形式がそれだけの価値を持つからであって、Racket が障壁だからではない。
- **決定的な依存関係**。ツールチェーンは Nix であり、二つ目は存在しない。Homebrew も conda もなく、デプロイ先のホストに「python のスパイス台所」を作らない。PDF レンダラは依存物であり、グラフ可視化ライブラリも依存物である。どちらも、誰かが本当に必要とするまでは断られてきた。
- **契約が先、バックエンドは差し替え可能**。永続する製品は SDK の契約、HTTP API、セキュリティモデル、そしてプロトコルである。二つ目の実装が狙うのはそれらであって、Racket のコードではない。生成されるリファレンスが存在するのは、契約をコード自身に書き記させるためである。
- **自前ホスティングとプライバシー優先**。運用者がモデルの URL を外部に向けない限り、バイト列がインスタンスの外に出ることはない。ブロブストアが組織の名前空間

の内側で内容アドレス化されているのは、まさに二つのテナントが互いのファイルについて何かを知ることが決してないようにするためである。

- **印をつけるのではなく拒否する**。完成したように見える誤った結果は、失敗したステップより悪い。翻訳の下書き生成、フィールド抽出、知識グラフの抽出は、いずれも検証を通らないモデルの応答を拒否する。これはあまりにも頻繁に現れるので、この本の中に独立した節を持っている。

1.3 どこに何があるか

```
docs/design/          設計ドキュメントと決定ログ
docs/reference/       生成物：ツール、ワークフロー、権限、API、プラグイン、SDK
docs/ops/             運用者向けランブック
docs/integrators-guide.md  自社製品をこのプラットフォームに載せる
docs/book/            この本
refimpl/racketmaximus/
  server/main.rkt     HTTP サーバ：ハンドラ、起動、プラグイン読み込み
  server/routes.rkt   宣言されたルート表
  domain/             モデル：authz, repo, flow, sched, quota, i18n, kg, ...
  pkgs/               db-kit, web-kit, cli-kit, sha2-kit：プラットフォームを知らないライ
    ブラリ
  plugins/            同梱プラグイン：doc-indexer, doc-pipeline, knowledge-graph, rs3,
    ...
  cli/                telemachus-localize, telemachus-docs, telemachus-worker,
                      telemachus-secrets
  static/             コンソール（HTML 一枚）とその英語文字列
  locales/            カタログ：en, ja, nl, es-419
  test/               ユニットスイート、スモークスクリプト、e2e ゲート、モックサーバ
```



Chapter 2

動かしてみる

2.1 ツールチェーン

```
nix develop                          # リポジトリのルートで。Racket 9.3 を固定し、
    PLTCOLLECTS を設定する
cd refimpl/racketmaximus
raco make server/main.rkt            # 事前コンパイル。これをしないと起動が遅い
raco test test/*-tests.rkt           # ユニットスイート
racket server/main.rkt               # http://127.0.0.1:8835
```

シェルに入るのはリポジトリのルートからであって、`refimpl/racketmaximus` の中からではない。シェルのフックは現在のディレクトリから `PLTCOLLECTS` を組み立てるので、一段下で入るとパスが二重になり、すべての `db-kit` の読み込みが「collection not found」で壊れる。

`raco test test/*.rkt` は決して実行しないこと。このグロブは `test/mock-*.rkt` を含んでしまうが、これは永久にブロックするモックサーバである。ユニットスイートは `test/*-tests.rkt` である。

2.2 環境変数

```
DATABASE_URL=sqlite:///$PWD/data/telemachus.db   # または postgres://user:pass@host:
    port/db
TMACH_MODEL_URL=http://127.0.0.1:11434/v1/chat/completions   # OpenAI 互換のエンドポイ
    ント
TMACH_MODEL=qwen2.5:7b
TMACH_HOME=login          # `beta` にするとオンボーディング用の導線を / で配信する
TMACH_S3_PORT=8836        # S3 エンドポイントを有効にする。二つ目のリスナーである
TMACH_MULTITENANT=1       # 一つのインスタンスに複数の会社を載せる
PORT=8835
```

これらはかつて `TELEMACHUS_*` という綴りだった。その綴りはいまも応える。プラットフォームは新しい名前を先に読み、なければ古い名前へ落ちる。期限はなく、移行すべきものもない。古い名前で書かれた配備はそのまま動き続ける。プラットフォームが読む変数の一覧は `docs/ops/configuration.md` にある。

`TMACH_MODEL_URL` がないと、プラットフォームは入力を大文字で返すだけの模擬モデルで動く。チャットは動き、スモークスイートも動き、本物の推論を必要とするものだけが動かない。モデルを必要とするツール — 翻訳の下書き生成、ドキュメントパイプライン、知識グ

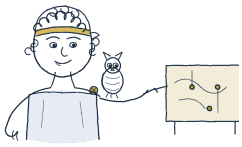
ラフの抽出 — は、誤解を招くメッセージで毎回の検証に失敗するのではなく、実行そのものを拒否する。これは意図的である。運用者が読む必要があるのは「モデルが設定されていない」であって、「応答が JSON ではなかった」ではない。

2.3 最初の起動

ユーザー名とパスワードを添えて `POST /api/bootstrap` を呼ぶと、運用者、最初の組織、最初のチームが作られ、運用者のトークンが返る。これはちょうど一度だけ成功する。その後は `/` から、あるいは `POST /api/login` でサインインする。

2.4 開いてみるファイル

リポジトリのルートにある `CLAUDE.md` が作業ノートである。午後を一つ潰したすべての落とし穴が、サブシステムごとにそこへ書かれている。`flake.nix` がツールチェーンであり、`refimpl/racketmaximus/config.rkt` が環境である。



Chapter 3

コードの地図

3.1 層

サーバはモジュール一つ、`server/main.rkt` であり、意図的に薄い。リクエストを解析し、プリンシパルを解決し、`domain/` を呼び、応答を符号化する。製品として間違いうるものはすべて `domain/` の中にあり、そこは HTTP を何も知らず、ユニットスイートが直接動かす対象である。

モジュール	担当
`domain/authz`	プリンシパル、ロール、権限カタログ、`can?`、付与、トークン、監査
`domain/db`	マイグレーション。起動時に適用され、方言に依存しない
`domain/repo`	ドキュメント保管庫：オブジェクト、バージョン、ブロブ、共有、来歴、トリガ、パイプラインのツール
`domain/flow`	ワークフローエンジン：仕様の検証、束縛、実行のリデューサ
`domain/sched`	ジョブスケジューラと同時実行の調停
`domain/quota`	使用量の台帳と上限
`domain/agent`	ツールレジストリ、エージェントループ、プラグインローダ
`domain/ai`	モデル実行：一回の呼び出し、ストリーム、あるいは模擬の代替
`domain/i18n`	カタログ、ICU 書式、lint、翻訳管理
`domain/kg`	知識グラフ
`domain/apps`	検索、翻訳 — 残りを組み合わせて作られたアプリケーション
`domain/orgs`	マルチテナンシー：チームの上にある組織
`domain/beta`	オンボーディングの導線

3.2 四つの小さなライブラリ

`pkgs/` には、TMach について何も知らず、そのまま外へ持ち出せるコードが入っている。

db-kit/portable は Racket の db をそのまま置き換えるもので、PostgreSQL では ? のプレースホルダを $n に書き換える。db ではなく必ずこちらを require すること。忘れても SQLite では何も起こらず、PostgreSQL で syntax error at or near "AND" として失敗する — そしてテストファイルも生の SQL を発行するので、この規則はテストにも当てはまる。

web-kit は JSON の制御プレーン向けに Racket の web-server を包み、さらに web-kit/http1 を加える。これはリクエストの本体をポートとして扱う HTTP/1.1 のリスナーである。二つ目が存在するのは、S3 エンドポイントが数ギガバイトのファイルを運ぶからであり、そして Expect: 100-continue に本体の最初の読み取り時点で遅延して応答しなければならないからである。そうすることで、読む前に拒否するハンドラはバイト列を一切受け取らずに済む。

cli-kit はコマンドラインツールの引数解析と、終了コードの契約である。ツールには「うまくいった」と「壊れた」は言えたが、ドメインが入力を拒否したとは言えなかった。これは別の答えであり、別の対応を要する — 入力を直すのか、運用者を呼び出すのか。いまは四つの結果がある。0 は実行され、結果が標準出力にある。2 は拒否で、常に理由を伴うエラーオブジェクトが返る。1 は実行できず、標準出力は空である。64 は CLI の誤用で、標準エラーに出る。このうち二つは文書化ではなく強制されている。理由のない拒否は却下される（それは 2 の終了コードを着た 1 である）。そして実行されなかった処理は標準出力に何も書かない。誰かに成果物を手渡してはならないからである。

sha2-kit は libcrypto の上の SHA-256 と HMAC-SHA-256 である。プラットフォームの知識を持たないので domain/ の中のモジュールではなくパッケージであり、FFI を使うので crypto.rkt — その契約は「ネイティブ依存なし」である — とは分けてある。正しさは NIST のベクタに固定されている。バインディングにおける危険は署名が間違っていることであり、ベクタはそれをただちに教えてくれる。

3.3 プリンシパル一つ、検査一つ

すべてのリクエストはプリンシパルに解決される。ユーザー ID、チーム ID、運用者フラグ、任意の組織ロール、そして API トークンと S3 キーについては、発行者ができたことに上限をかけるスコープの一覧である。プラットフォームにおけるすべての認可判断は一つの関数 can? であり、次章で説明する。ハンドラは require-perm を呼ぶ。拒否は例外を上げ、サーバがそれを翻訳済みの 403 に変える。

3.4 開いてみるファイル

server/main.rkt（require 群と HANDLERS の表を流し読みする）、domain/authz/authz.rkt、pkgs/db-kit/portable.rkt。



Chapter 4

認可のモデル

4.1 権限とロール

権限とは `resource:action` という形の文字列である。ロールはそれらに名前を付けた集合であり、組み込みのチームロールは owner、admin、member、viewer の四つ、その上に二つの組織ロールが乗る。ワイルドカード（`files:*`、`*:read`、`*:*`）もあるが、チームロールが何を付与しようと到達できない層が二つある。`instance:*` は運用者だけのものであり、`org:*` は会社の組織ロールだけのものである。すべての権限は宣言のそばに一行の説明を持つ。説明のない権限を付与する組み込みロールは、読み込み時に失敗する。生成される `permissions.md` がカタログとロールの対応表を示す。

4.2 `can?` の順序

1. **組織のゲート**。リソースが別の組織のものであれば、無条件に拒否する。他の何かを参照する前である。これがマルチテナンシーを健全にしている。共有も、付与も、スコープも会社の境界を越えて潜り込むことはできない。それらを尊重するはずの検査が、そもそも走らないからである。

2. **層**。 `instance:` の権限は運用者フラグを必要とし、`org:` の権限は組織ロールを必要とする。

3. **ロール、あるいは付与**。プリンシパルのチームロールがその権限を含んでいなければならない — あるいは、そのリソースについてその権限を名指すリソース付与を持っていなければならない。付与とは委譲である。編集可能なドキュメントを viewer に共有すれば、その viewer はそのドキュメント一つを編集できる。member に `manage` の権限を渡せば、その人はさらに先へ共有できる。共有の実装より前は、付与は到達可能性を広げるだけで、ロールに欠けている権限を与えることはできなかった。それでは「編集のために共有する」が嘘になっていた。

4. **所有者は可**。リソースの所有者は、そのリソースについてチーム層のすべての権限を持つ。同僚のドキュメントを削除できない member も、自分のものは削除できる。

5. **スコープ**。 API トークンや S3 キーは、その発行者に上限をかける。スコープの一覧もまたその権限を含んでいなければならない。スコープは狭めるだけで、広げることは決してない。

6. **到達可能性**。非公開のリソースに到達できるのは、その所有者、運用者、そして付与を受けた者である。チームに公開されたものにはそのチームの誰でも到達でき、共有されたものには所有者と付与を受けた者が到達できる。

4.3 権限のまとまり

ドキュメントを共有する人は、権限の文字列を選ぶわけではない。閲覧・編集・管理のいずれかを選び、プラットフォームがそれに対応する付与の行を書く。閲覧は `files:read`、編集はそれに `files:write` を加えたもの、管理はさらに `files:delete` と `files:manage` を加えたものである。ワイルドカードは決して使わない。付与の行は、書かれているとおりのことだけを意味する。さらに先へ共有できるのは管理だけである。付与は同じ組織のユーザーまたはチームを名指すことができ、有効期限（エポック秒。`BIGINT` である。32 ビットの列が PostgreSQL のスモークで 2099 年の期限に桁あふれした）を持つこともできる。期限切れの行は無視されるが、削除はされない。誰に何が与えられたかの監査記録そのものだからである。

4.4 マルチテナンシー

既定では無効である。フラグを立てると、チームの上に組織が乗り、スーパー管理者がインスタンスを運営し、組織管理者が一つの会社を運営する。組織管理者は管理はするが読まない。`documents:read` もなく、`chat:use` もない。チームのデータを必要とする会社の管理者は、そのチームに member として参加し、その参加は監査される。手順 0 のゲートは機能フラグを読まないので、データの入ったインスタンスでフラグを下ろしても、管理用の面が隠れるだけで強制は続く。

4.5 開いてみるファイル

`domain/authz/permissions.rkt`、`domain/authz/authz.rkt`（`can?` と `has-grant?` を読む）、`docs/design/rbac-and-teams.md`、`docs/design/document-sharing.md`、`test/authz-tests.rkt`、`test/tenancy-tests.rkt`。



Chapter 5

データとマイグレーション

マイグレーションは `domain/db/migrations.rkt` にあり、起動時に適用される一つのリストである。どの文も方言に依存してはならない。いまは SQLite、二つ目の対象は PostgreSQL であり、ユニットスイートと二つのスモークスイートはあらかじめ設定された `DATABASE_URL` を尊重するので、すべてがどちらに対しても走る。何かを完了と呼ぶ前に PostgreSQL で確かめること。この本の著者たちはそうしており、それによって int4 の桁あふれ、プレースホルダの書き換え漏れ、そしてすべての S3 一覧表示を壊していたタイムスタンプ形式を捕まえた。

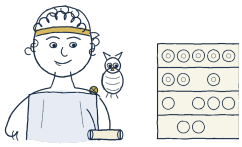
その実行から得られた規則のいくつか。

- 予約語は引用符で囲む（`"window"`）。
- 期間はタイムスタンプの演算ではなく、エポックの整数列で表す。`CURRENT_TIMESTAMP` は秒の解像度しかなく、その文字列表現は方言によって異なる。一秒の中で「新しい順」に並べなければならないものは、自分専用のエポック列を持つ。監査ログはこれを遅くに学んだ。文字列のタイムスタンプで並べ、次にランダムな UUID で並べていたので、同じ秒に書かれた二つのエントリは、実際に起きたこととは無関係な順序で返ってきた — レビューの判断が、それを引き起こした評価より前に読まれることもありえた。ここで足りなかったものにも注意してほしい。ミリ秒である。書き込みの集中は単位が何であれ一つの刻みの中に収まるので、列はマイクロ秒であり、かつ書き手がプロセス内で厳密に増加するよう値を押し上げる。時計の読み取りは並び順の鍵ではない。
- upsert には `ON CONFLICT ... DO NOTHING` または `DO UPDATE` を使う。どちらの方言も、衝突対象を名指す構文を解する。
- `hasheq` の反復順序は Racket のプロセスをまたいで安定しない。再現可能でなければならないもの — ディスクに書く JSON、生成される表の列の順序 — に、それを決して頼らないこと。

ユニットテストの土台は `test/db-fixture.rkt` にある。`(fresh-db)` は、マイグレーション済みで他から隔離されたデータベースを返す。SQLite ではメモリ上に、PostgreSQL では土台ごとに専用のスキーマとして作られる。テストファイルは並行して走るからである。

5.1 開いてみるファイル

`domain/db/migrations.rkt`、`test/db-fixture.rkt`。



Chapter 6

ドキュメント保管庫

6.1　オブジェクト、バージョン、ブロブ

保管庫のオブジェクトは所有できるリソースであり、ノートが持つのと同じ三つのフィールド — チーム、所有者、公開範囲 — を持つ。だから新しい認可コードなしに `can?` がそれを統べる。同じキーに二度書き込むと、一方を置き換えるのではなくバージョンが追加される。付与や URL が指すのは、安定した存在であるオブジェクト ID である。バイト列は SHA-256 を鍵とする内容アドレス方式のストアに置かれ、組織ごとに名前空間が分かれている。テナントをまたぐ全体的な重複排除は、存在を漏らす神託になってしまう。ストアにプリンシパルが渡されることは決してない — 名前空間とダイジェストだけである — ので、ストレージのバックエンドには間違えるべき認可が存在しない。

アップロードは生の `PUT` の本体であり、JSON の中の base64 では決してない。ダウンロードはすべて添付ファイルとして、`nosniff` と拒否する CSP を付けて配信される。短いインライン許可一覧に載っている型だけが例外である。SVG と HTML が決してインラインにならないのは、それらがコンソールとオリジンを共有するからである。

6.2　S3 エンドポイント

二つ目のリスナーが、`aws`、`rclone`、`boto3`、Cyberduck が動くだけの S3 を話す。SigV4 は仕様から実装され、AWS の公表するベクタに固定されている。バケットはパス形式で、チームのスラグが名前になる。範囲指定の GET（必須である — CLI は大きなオブジェクトを並行した範囲指定で取得する）、マルチパートアップロードにも対応する。アクセスキーはスコープを持つ通常の資格情報であり、秘密鍵は平文で保存される。SigV4 が検証のためにそれを必要とするからである。実装していないサブリソースは 501 を返し、黙って成功を装うことは決してない。署名付きリンクは呼び出し元自身の最新のキーで署名されるので、そのキーを失効させればリンクも死ぬ。

6.3　共有と来歴

共有は前章の権限モデルそのものであり、すでに存在していた付与のテーブルの上に乗っている。来歴は `repo_derivations` である。ワークフローがあるドキュメントから別のドキュメントを書くとき、出力は作成の瞬間における元の公開範囲と有効な付与を受け継ぐ — 元が非公開なら最初のバイトから非公開である — そして派生の行が、元のバージョン、実行、

ステップを名指す。それ以降、二つのドキュメントは互いに独立である。「この書式はどこから来たのか」はクリック一つで分かる。

6.4 開いてみるファイル

`domain/repo/repo.rkt`、`domain/repo/blobs.rkt`、`plugins/rs3/`、`domain/s3/sigv4.rkt`、`docs/design/document-repository.md`、`test/repo-tests.rkt`、`test/s3-smoke.sh`。



Chapter 7

ツールとエージェント

7.1 ツールを宣言する

```
(define-tool doc_text
  #:description "Return the text of a repository document."
  (object string #:description "The repository object id")
  (version string #:optional #:description "A specific version id"))

(register-tool! "doc_text" doc_text "files:read"
                (lambda (conn principal args) ...))
```

`define-tool` は、モデルが見る OpenAI 互換の関数スキーマへ展開される。だから宣言と契約は一つのものである。ハンドラはデータベース接続、呼び出し元のプリンシパル、そして解析済みの引数を受け取る。自分の権限を自分で検査し、自分の AI 消費を自分で計量する。文字列でない結果は、エージェントの境界で JSON に符号化され、ワークフローの中では値のまま保たれる — リストを返すツールは `map` ステップにそのまま流れ込む。

7.2 レジストリ

組み込みツール、プラグインのツール、ワークフローのツールは、すべて一つのレジストリに収まり、出自の印が付く。チームはどのツールも無効にできる。エージェントは有効なツールだけをモデルに提示し、ディスパッチは無効なものを拒否する。プラグインはプラットフォームの権限でインプロセスに動く — `plugins/` に置くことがその同意である — そしてサンドボックス化されたプロセス外のプラグインは、別の、権限を限定した仕組みとして存在し、標準入出力の上で改行区切りの JSON を話す。

プロセス外のツールは、テキストだけでなく構造で答えることもできる。その結果メッセージは、人間向けのものと並んで任意の構造化フィールドを運ぶ。それがないと、そうした答えはワークフローが束縛する時点でみな JSON の文字列になってしまい、後続のステップには、プラットフォームがいま文字列化したものを解析するためだけに存在するインプロセスの橋渡しが必要になった — 一方でインプロセスのツールは、何スライスも前から値を返す自由を持っていた。プラットフォームは、文字列が本当は JSON のつもりだったのだと推測することは決してしない。プラグインが構造化フィールドを送ることで、そう言うのである。

7.3 モデルとの継ぎ目

モデルを呼ぶものはすべて `domain/ai/executor.rkt` の `run-chat` を通る。そして構造化出力を必要とするツールは、モデルをパラメータとして受け取る（`current-doc-chat`）。ユニットテストはそれに台本を与える。HTTP のスモークは `test/mock-llm.rkt` を使う。これは決定的な OpenAI 互換のサーバで、チャットモードでは `needle: reply` の対を集めたファイルから答え、最も長い needle が勝つ。モデルの URL が設定されていれば、同じスモークが本物のモデルに対して走る。こうして、モデルを必要とするパイプラインが CI では数秒で試験され、開発者のマシンでは qwen2.5:7b に対して検証される。

7.4 メッセージが運べるもの

メッセージの内容は文字列またはパーツのリストである。後者はゲートウェイやマルチモーダルのサーバが送ってくる、OpenAI 互換の `[{type: "text", text: ...}]` という形である。これをすべての経路 — エージェントループ、チャットのエンドポイント、ストリーム、そして pull の通信路 — のために一つのモジュールが決めているので、それらが食い違うことはありえない。テキストのパーツは連結される。パーツを運んでいて、しかもテキストがない内容は、静かな空文字列ではなく、名前の付いたエラーになる。その区別こそがすべてである。空の返答を作り出してはならない。何も言わなかったモデルと区別がつかなくなるからである。内容が存在しない、あるいは空であることは、依然として正当に「空」である — ツール呼び出しだけのターンがまさにそう見える — ので、エージェントの解析器が例外を上げるのは、そのターンにツール呼び出しも無いときだけである。これは一年にわたって静かなバグだった。`(if (string? c) c "")` がすべての返答を読んでいたので、パーツで答えたプロバイダは空のメッセージを生み、誰も文句を言わなかった。

呼び出し側は形を要求することもできる。`response_format` — `text`、`json_object`、`json_schema` — はプロバイダへ送られ、かつ返答がここで検査される。ほとんどのローカルサーバはこのフィールドを完全に無視するので、そうしなければスキーマの要求に対して散文が返ってきて、呼び出し側はそれを適合したものとして解析してしまう。検証に使うのは、ドキュメントパイプラインの抽出と同じ部分集合の検証器であり、同じ `$.total: expected number, got string` という拒否である。モデルのない模擬の代替は、大文字化した反響を適合した JSON と偽るのではなく、形の要求を拒否する。ストリーミングはストリームが終わった時点で形を判定し、最後のイベントでその判定を報告する。答えがすでに出て行った後で例外を上げるほうが、より悪い。

7.5 開いてみるファイル

`domain/tools/dsl.rkt`、`domain/agent/registry.rkt`、`domain/agent/plugins.rkt`、`domain/ai/executor.rkt`、`docs/reference/tools.md`。



Chapter 8

ワークフロー

8.1 仕様が契約である

ワークフローとは検証済みのデータ文書である。スラグ、型の付いた入力、そしてツールを使う・述語で分岐する・リストの上に展開するステップからなる。`define-workflow` はその文書を出力するマクロであり、API に POST された JSON 文書も同じ検証器を通る。二つ目の経路は存在しない。未知のフィールドは拒否される。これには、より新しい仕様バージョンや、このビルドに無いステップ種別も含まれる。束縛の下位言語は凍結されている — 参照と七つの述語だけで、算術も評価もない — そして逃げ道は「ツールを書く」ことである。

```
(define-workflow process-upload
  #:input ([object_id string] [schema object] [template string] [locales array])
  (step text     (tool doc_text #:object (in object_id)))
  (step fields   (tool doc_extract_fields #:text (out text result) #:schema (in
    schema)
                                          #:object (in object_id) #:run (run-id) #:
    step "fields")
                 #:retry 1)
  (step has_form (choice (empty (in template)) #:then translate_source #:else form))
  (step form     (tool doc_render #:template (in template)
                                  #:data (out fields result fields) ...))
  (step translate_form (map #:over (in locales)
                            (tool doc_translate #:object (out form result object_id)
                                                #:locale (item) ...))
        #:end)
  ...)
```

8.2 実行はリデューサである

エンジンはメモリに何も保持しない。`flow-advance!` はデータベースから実行を読み、その次のステップをスケジューラのジョブとして投入する。ジョブが終わると、そのステップの出力が書かれ、カーソルが進む。したがって永続性、キャンセル、クォータの受け入れ判定、チームごとの同時実行上限、組織のゲートは、作り直されるのではなくスケジューラから受け継がれる。展開は親の行一つと N 個の子ジョブであり、最後に終わった子がロックの下でカーソルを進める。そうしなければ、二つの子がそれぞれ次のステップを投入してしまう。

宣言された入力は必須であり型が付いている。一つでも欠けた実行は、三ステップ先で束縛

が null に解決されるのではなく、開始時点で拒否される。プラグインが提供するワークフローは、最初に参照された時点でチームの定義へ実体化される。その挿入が冪等なのは、コンソールが二つの参照を並行して投げるからであり、二つ目が一度 500 を返したことがある — これを見つけたのは端から端までのゲートであって、一つずつ参照するどのスモークでもなかった。

8.3　開いてみるファイル

`domain/flow/spec.rkt`（規範である）、`domain/flow/dsl.rkt`、`domain/flow/bind.rkt`、`domain/flow/run.rkt`、`docs/design/workflow-engine.md`、`docs/reference/workflows.md`。



Chapter 9

ドキュメントパイプライン

9.1 最初の利用者の道筋

チームがファイルをアップロードし、ワークフローがそれを処理する。テキストを抽出し、JSON スキーマに照らして構造化されたフィールドを取り出し、書式のテンプレートを埋め、結果を翻訳する。四つの中核ツールが組み合わさって、同梱の `process-upload` ワークフローになる。スキーマ、テンプレート、ロケールは入力なので、一つのワークフローが新しいコードなしに請求書と申込書の両方に仕える。すべての出力は元のドキュメントのそばに置かれる保管庫のドキュメントであり、元の付与と来歴の行を伴う。

9.2 印をつけるのではなく拒否する

フィールド抽出器はモデルの返答をスキーマに照らして検証し、必須フィールドの欠落、数値を求めた場所の文字列、そして発明された鍵を拒否する — `additionalProperties` を持たないオブジェクトのスキーマは閉じている。これは JSON Schema の既定からの意図的な逸脱である。テンプレートの描画器は、データが満たせないプレースホルダを空欄として描くのではなく拒否する。合計欄が空のまま完成したように見える書式こそ、この設計全体が防ぐために存在する失敗である。ステップは一度だけ再試行する。スキーマの不一致はモデルの間違いであり、温度ゼロで二度目に読ませれば多くの場合は適合する。

9.3 テンプレート

Markdown と HTML のテンプレートは `{{field}}`、ドット区切りの経路、そして `{{#each items}}` のブロックを使う。DOCX のテンプレートは zip であり、その `word/document.xml` が同じプレースホルダを運ぶ。これが聞こえるより難しい理由が二つある。Word は、著者が手を止めた瞬間や綴り検査が覗いた瞬間に、プレースホルダを複数の run に分割してしまう。だから `{{` とその `}}` のあいだにあるタグはすべて、描画の前に取り除かれる。そしてもう一つ、明細は項目ごとに表の一行を欲しがるので、本文が `{{#each items}}` だけの行がブロックを開き、`{{/each}}` だけの行がそれを閉じる。PDF の描画は見送っている。PDF の描画器はどれも依存物であり、そもそも人が編集するのは DOCX か HTML の書式である。

9.4　トリガ

トリガとはチーム単位の購読である。この接頭辞とこれらの内容型に合致するドキュメントが届いたら、それを渡してあのワークフローを走らせる、というものである。トリガが評価される場所はちょうど一つ — `repo-put!` の末尾にあるフック、バージョンが確定した後である — ので、コンソールからのアップロード、テキストドキュメントの互換層、S3 の `PUT` が構造上すべて覆われる。合致すると、アップロードした人として、そのスコープもすべて含めて実行が投入される。実行が読めるのはアップロードした人が読めたものだけであり、その出力は実在する誰かが所有する。発火はバージョンごとにちょうど一度である。パイプライン自身の出力が、それを生んだトリガを再発火させることは決してなく、参加を表明したトリガが、自分が始めた実行の出力で発火することも決してない。この規則がないと、参加を表明したトリガはテストの中で、流出制限が止めるまでに自分自身を五十回走らせた。既定の `files:*` スコープで発行された S3 キーはワークフローを開始できない — トリガの履歴がそうはっきり言う — 発火させたい接頭辞には `workflows:run` が必要である。

9.5　開いてみるファイル

`domain/repo/doc-tools.rkt`、`domain/tools/jsonschema.rkt`、`domain/repo/triggers.rkt`、`plugins/doc-pipeline/main.rkt`、`docs/design/document-workflows.md`、`test/doc-pipeline-tests.rkt`、`test/doc-triggers-tests.rkt`、`test/doc-pipeline-smoke.sh`。



Chapter 10

翻訳管理

10.1　三つの住所

サーバの拒否メッセージは `surface/messages.rkt` にある。コンソールの外枠は `static/ui-strings.json` に、生成されるドキュメントの散文は `docs/reference/strings.json` にある。この三つはいずれも一つの抽出器から見た面であり、抽出器はそれらを基底カタログ `locales/en.json` へ畳み込む。翻訳はその隣に置かれ、各文字列はそれが作られたときの英語のハッシュに固定されている。英語を編集すれば、どの言語の翻訳も自動的に古くなる。状態を書き換えるものは何もない。導線にある運用者が書いた文面は例外で、カタログではなく体験文書に重ねる層である。それはコードではなく内容だからである。

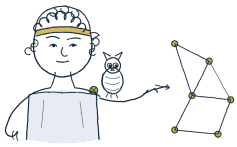
解決はリクエストのロケールヘッダ、次にインスタンスの既定である。未知のロケールは英語ではなくインスタンスの既定へ落ちる。日本語を既定とするインスタンスが成り立つ。

10.2　翻訳マネージャ

旗艦である。プラットフォームが自分自身のプリミティブから本物のツールを建てている。ディスク上のカタログは出荷される成果物のままであり、二つのテーブルがその周りのワークフローである。未訳と陳腐は導出されるものであって、保存されることは決してない。翻訳者は自分の訳を自分で承認できない。AI による下書きはスケジューラのジョブの列であり、一つにつき二十文字列、クォータで計量される — そして一括の下書きはチームが予算を超えた時点で止まる。それは停止したように見えるが、プラットフォームが正しく働いている姿である。プレースホルダを失う、発明する、壊す下書きは拒否される。検査は波括弧を数える。書式化器は意図して寛容であり、拒否すべき下書きをちょうど受け入れてしまうからである。オランダ語とラテンアメリカのスペイン語はこのツールで作られ、全文字列がレビューされ、ツール自身のゲートを通っている。

10.3　開いてみるファイル

`domain/i18n/`、`cli/telemachus-localize.rkt`、`locales/`、`docs/design/localization.md`。



Chapter 11

知識グラフ

実体と、それらのあいだの関係を、チームのドキュメントから抽出する。すべての事実は、それを主張したドキュメントとバージョンまで辿れる。出典なしにグラフへ入るものはなく、その一つの規則から残りが導かれる。

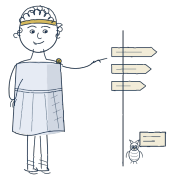
公開範囲は継承されるもので、割り当てられるものではない。ある事実がプリンシパルに見えるのは、その言及のうち少なくとも一つが、その人が読めるドキュメントの上にあるときである。言及は行ごとに絞られるので、非公開の覚書の抜粋は、チームが報告書を通してその事実を見られる場合でも非公開のままである。呼び出し元がどの言及も見られない実体は、その人にとって存在しない。そして API はそれを、存在しない ID と区別しない。

検証が来歴である。抽出器はモデルに厳密な形を求め、本文の逐語的な部分文字列ではない抜粋、同じ返答の中に無い実体への関係、名前のない実体を拒否する。ドキュメントの新しいバージョンは、その古い言及に取って代わる。ドキュメントを削除すればその事実は忘れられる。言及が一つも残らなかった実体や関係は刈り取られる。

意図して狭くしてある。重複排除は型と正規化した名前によって行い、型をまたぐ統合はしない。そこは知識グラフが死に行く場所である。グラフの問い合わせ言語はない — 名前の付いた実体から二ホップあれば、人が実際に抱く問いは覆える — 可視化ライブラリもない。タブは出典付きの一覧であり、エージェントの `kg_query` ツールは、そう述べたドキュメントを添えてテキストで答える。

11.1　開いてみるファイル

`domain/kg/kg.rkt`、`domain/kg/kg-tools.rkt`、`plugins/knowledge-graph/main.rkt`、`docs/design/knowledge-graph.md`、`test/kg-tests.rkt`。



Chapter 12

ルート表と生成されるリファレンス

12.1 制定ではなく宣言

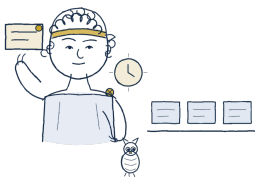
すべての HTTP ルートは `server/routes.rkt` の一つのエントリである。メソッド、経路のパターン、ハンドラの鍵、どう認証するか、何の権限を強制するか、どの機能フラグの後ろにいるか、そして一行の説明である。サーバは一つの表で鍵をハンドラに結び付け、どちらかの側が相手に無いものを名指していれば起動を拒否する。説明のないエンドポイントは存在できず、説明のあるエンドポイントが存在しないこともありえない。公開かどうかは `cond` の節のそばのコメントではなく宣言であり、だからこそテストがそれを検査できる。

12.2 生成され、ゲートされる

`telemachus-docs describe` は、ツールレジストリ、プラグインローダ、ワークフロー仕様、権限カタログ、ルート表を評価して一つの JSON モデルにする。`render` が `docs/reference/` を書き、`check` が再生成して一バイトでも違えば失敗する。出力はコミットされるので、ツールの引数を変えるプルリクエストは、同じ差分の中にドキュメントの変更を示す。すべての説明はカタログの中の `doc.*` メッセージでもあり、他のすべてと同じマネージャで翻訳される。

```
racket cli/telemachus-docs.rkt render      # docs/reference/ を再生成する
racket cli/telemachus-docs.rkt check       # CI のゲート
racket cli/telemachus-docs.rkt render --locale ja
```

したがってエンドポイントを追加するのは、表に一つのエントリ、対応表に一つのハンドラ、一度の render、一度のコミットである。render を忘れれば CI が落ち、どちらかの半分を忘れれば起動が拒否される。



Chapter 13

ジョブの待ち行列

13.1　ジョブとは何か

ジョブとは一つの行である。チーム、ユーザー、種別、JSON のペイロード、優先度、状態。実行中のジョブについてメモリに置かれるものは何もない。だから再起動で仕事が失われることはなく、だからワークフローのステップ、一括翻訳、別のマシンへ回されたモデル呼び出しが、すべて同じものでありうる。上限のあるワーカースレッドのプールが待ち行列を汲み出す。各スレッドは一つのジョブを確保し、その種別のハンドラを走らせ、結果をその行に書き戻す。

二つの方針は、ハンドラの内側ではなく確保の時点で適用される。だからどの種別もそれを忘れることができない。一つは**チームごとの同時実行上限**で、あるチームの一括処理がプールを独占するのを止める。もう一つは**クォータの受け入れ判定**で、予算を超えたチームのジョブを失敗させるのではなく待ち行列に留める。何も実行されていないのに動きの止まった待ち行列は、たいていは AI 予算を使い切ったチームであり、それはプラットフォームが正しく働いている姿である。

13.2　一つを確保する

確保は些細に見えるが、そうではない。プールは待ち行列の行を走査し、候補を一つ選び、実行中へ切り替える — しかしその切り替えは、その行がまだ待ち行列にあることを条件とする必要があり、そして条件は誰かが結果を読まなければ無価値である。

```
UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP,
                lease_until=?, executor_id=?, claim_token=?
 WHERE id = ? AND status='queued'
 RETURNING id
```

確保は、行が返ってきたときにだけ信じられる。これは見かけより重要である。ドライバは PostgreSQL では影響した行数を報告するが SQLite では報告しないので、行数は方言に依存する答えになってしまう。一方 `RETURNING` はどちらでも動く。更新が何にも合致しなかった確保者は、別のワーカーが握っているジョブを走らせるのではなく、次の候補へ移る。これより前は、排他は一つのプロセスの中のロックに頼っていた — プロセスが二つになるまでは問題なかった。

13.3　ワーカーが死ぬとき

すべての確保は**リース**を伴う。保持者が仕事をしているあいだ更新し続けるタイムスタンプである。保持者が死ねばリースは失効し、回収役がそのジョブを試行回数を一つ増やして待ち行列へ戻す。三回を過ぎれば本当に失敗させる。これはインプロセスのプールにも、遠隔のワーカーとまったく同じように当てはまる。そうでなかったころは、プロセスが死んだプールのジョブが永久に `running` のまま座り、そのチームの同時実行枠を食い続けていた。古いビルドが残したジョブ — 実行中で、リースのないもの — は、プールの起動時に待ち行列へ掃き戻される。

回復はより微妙な問題を生む。そしてそれが覚えておく価値のあるものである。元の実行がまだ生きているうちにリースが失効したとしよう。回収役がジョブを待ち行列に戻す。同じ保持者がもう一度それを確保する。最初の実行がついに終わり、結果を書く。状態は一致する。保持者も一致する。どちらの検査も、二つの試行を見分けられない。そこで、確保はそのたびに**隔離トークン**を鋳造し、その試行についての後続のすべての書き込みがそれを運ぶ — 最終的な書き込み、リースの更新、そして遠隔ワーカーの心拍、完了、失敗である。待ち行列への差し戻しはその列を消し、最終的な書き込みもすべて消す。前の試行のトークンが再び一致することは決してなく、現在の試行からの二度目の書き込みも一致しない。

```
claim  -> id + lease_until + claim_token
heartbeat / complete / fail  -> その claim_token を運ばなければ 409
```

13.4　仕事の側へやって来る実行基盤

推論ホストが、サーバから呼び出せる場所にいる必要はない。**pull 型の実行基盤**は、自分が走らせられるジョブを長いポーリングで待ち、走らせているあいだ心拍を送り、結果を投稿するループを回す — だから NAT の後ろや tailnet の上からでも、外向きの接続だけで働ける。その資格情報は、スコープが `jobs:execute` だけの API トークンであり、一度だけ表示され、実行基盤の行に結び付けられる。実行基盤はそのトークンを通して解決され、リクエストの中のフィールドを通して解決されることは決してない。

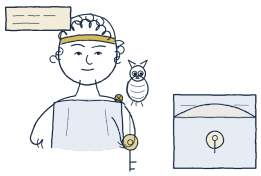
遠隔の種別は、インプロセスのハンドラなしで登録される。プールはそれを飛ばし、ワーカーだけがそれを確保でき、その結果はその種別が供給しなければならない関数によって検証される。代わりに厳しくしてくれるローカルのハンドラが存在しないからである。同梱されているのは `infer.chat` — まさに `run-chat` が話す通信路である — なので、同期的なモデル呼び出しは子ジョブになってワーカーを待ち、モデルを使うすべてのツールは、そのチャットが別の場所へ回されても変わらずに動く。子ジョブは親を運び、確保の時点でチームの上限から除外される。そうでなければ、上限が二の環境で二つの親が、誰も確保できない子ジョブをそれぞれ永久に待つことになる。

13.5　どのモデルが何を走らせるか

モデルの役割は実行基盤の名前へ解決される。まずチームの設定、次に環境変数、最後にローカルのモデルである。同梱されている役割は一つ — `utility` — で、これが量のある仕事を担う。知識グラフの抽出、パイプラインのフィールド抽出と翻訳、そして翻訳マネージャの下書きである。人のチャットがそこへ行くことは決してない。ツールは自分のモデル呼び出しの周りでこの役割を設定するので、GPU の箱は、どのツールもその存在を知らないままに一括処理を引き受けられる。存在しない実行基盤を役割に設定することは拒否される。そうでなければ、打ち間違いがすべての抽出を静かにローカルのモデルへ送り返してしまう。

13.6 開いてみるファイル

domain/sched/scheduler.rkt（claim-job!、finish!、with-lease を読む）、domain/exec/pull.rkt、domain/ai/roles.rkt、cli/telemachus-worker.rkt、docs/design/pull-executors.md、test/pull-tests.rkt、 test/pull-smoke.sh。



Chapter 14

秘密と玄関

14.1　そもそも誰が呼んでよいのか

これらすべての前に、リクエストはプリンシパルにならなければならない。そして長いあいだ、それはあまりに容易になれた。ベアラトークンがないとき、サーバは二つのヘッダ — `X-Telemachus-User` と `X-Telemachus-Team` — に落ち、それをそのユーザーへ解決していた。ノートパソコンの上ではこれは便利である。スクリプトがトークンを鋳造せずに誰かとして振る舞える。しかしネットワークのアドレスに結び付けられたインスタンスでは、これはログインであり、しかも無料のログインだった。そのネットワーク上のどの呼び出し元も、自分が名乗ったとおりの誰かであり、そのユーザーが持つすべての権限を持った。これがどこに位置するかに注意してほしい — ロールの検査、スコープ、組織のゲート、セッション方針の前である。そのどれも、プリンシパルが存在するまで走らない。その層での身元の誤りは、その上のどの制御でも捕まえられない。

いまヘッダが尊重されるのは信頼された相手からだけである。ループバック、あるいはリバースプロキシが本当にそこで終端しているために運用者が `TMACH_TRUSTED_HEADER_PEERS` で名指したアドレスである。私的に見えることを理由に信頼されるものは何もない — 「10.x は内部だ」は、他人のネットワークについての仮定である。`POST /api/bootstrap` も同じように門を構えている。初期化されていないインスタンスは最初に呼んだ者のものになり、LAN の上ではそれは他人だからである。ループバックに到達できない運用者は、代わりに秘密を提示する。

この規則は直接試験できるよう、それ自身の小さなモジュールに住んでいる。これは聞こえるより重要である。どのスモークもサーバと同じホストで走るので、スモークが他人であることは決してできず、この穴は一年分のスモークを生き延びた。

14.2　何がハッシュ化され、何ができないのか

パスワードと API トークンはハッシュとして保存される。トークンは環境に置いた胡椒を鍵とする HMAC-SHA-256 であり、プロセスの環境を伴わないデータベースのダンプでは再現できない。胡椒を回せば全員が一度にサインアウトする。これは立派な緊急手段であり、回せるようにしてある理由でもある。トークンはいまや期限も持つ。セッションは三十日、コンソールが発行した API トークンは九十日。期限のない行は依然として「無期限」なので、更新によって誰かがサインアウトすることはない。

三つの秘密はハッシュ化できない。比較されるのではなく使われるからである。TOTP の種はコードを生成する。S3 の秘密鍵は署名を検証する HMAC の鍵になる。push 型の実行基

盤の鍵は、自分のリクエストに署名する。この三つはかつて平文で置かれていた。だから一つのダンプ — バックアップ、レプリカ、チケットに貼られた pg_dump、盗まれたボリュームのスナップショット — があれば、そのすべてを再現できた。

14.3　残りを封じる

それらは AES-256-GCM で封じられ、鍵は環境から来る。データベースから来ることは決してない。この主張は意図して狭く、そして正確に述べる価値がある。コードにあった以前の注記が半分だけ正しかったからである。これは**ダンプ**を守り、**ホスト**に対しては何もしない。サーバとして動いている攻撃者は、自分自身の環境から鍵を読める。しかしダンプはその環境を伴わずに旅をする。そして本来あるべきでない場所へたどり着くのは、ダンプのほうである。

```
enc:v1:<key-id>:<nonce>:<ciphertext+tag>
```

この接頭辞を持たないものは平文であり、そのまま読まれる。だから鍵の採用はマイグレーションではなく、停止時間も要らない。行は書き換えられるときに封じられ、残りは CLI が封じる。同じ CLI が回転も行う — 以前の鍵が読み、現在の鍵が書く — そしてこれは意図してサーバが起動時にすることではない。起動時に打ち間違えた鍵は、誰も持っていない鍵ですべての行を封じてしまう。鍵をまったく使わずに動かすことも引き続き支持される。それが「データベースが信頼境界である」という立場であり、変わったのは、それが静かに受け継がれるのではなく、起動時の一行と管理の状態表示に見えるようになったことだけである。

二つの細部が重みを持つ。各値は、自身の列と行の ID を追加の認証データとして封じられる。だから一つの行から持ち出して別の行へ貼った暗号文は開かない — 書き込み権限を持つ者が、既知の秘密を別のプリンシパルの上へ移すことはできない。そして開かない封印された値は例外を上げる。空文字列へ劣化してはならない。空の TOTP の種は、誰かの二要素認証を静かに無効にしてしまう。

14.4　二要素認証と、戻る道

パスワードは変更でき、アクセスキーは再発行できる。TOTP の種は、それを使うことによって回転させることができない。だから漏れた種は永久のものになっていた — そこで、取り消せるようにした。ユーザーは自分のものを切ることができ、運用者は誰のものでも失効させられる。どちらも監査される。これで漏洩は覆える。失くした電話は覆えないので、登録のときに一度しか使えない**回復コード**を十個渡す。コードは TOTP のコードが通る場所ならどこでも通る。種と違い、コードは比較されるので、トークンと同じようにハッシュ化される。

価値があるのは小さな決定のほうである。照合は大文字小文字とハイフンを許す。そして文字の集合からは人が混同する文字を落としてある。これらは声に出して読まれ、紙から打ち込まれるものだからである。コードを使うのは、それを照合するのと同じ一文である。だから同時の二つのサインインが同じコードを使うことはできない。使われた行は削除されずに残る。だから「すでに使われた」が「発行されていない」と区別できたままになる。再発行は集合ごと置き換える。紙の上の一揃いは、そのアカウントを開くものについての全部の真実であるべきだからである。そして二要素認証の取り消しは、コードも一緒に削除する。

14.5　セッションはどれだけ続くか

二つの時計がある。別々の攻撃に答えるからである。**無操作**のタイムアウトは、放置された机を限る。**絶対**の上限は、使われている盗まれたトークンを限る。そちらは決して無操作にならない。同梱の既定値は三十分と八時間 — OWASP 自身の帯のうち、もっとも支障の少ない端である — そしてどちらかに `null` を置けば、その軸に上限はない。これを明示的に綴ってあるので、「設定しなかった」と「切った」が同じに見えることはありえない。

統べられるのは**セッション**だけである。マシン用のトークンは設計上ずっと無操作であり、それを無操作で切るのは静かな障害である。だから API とワーカーのトークンは除外される — そしてトークンがどちらであるかを言うパラメータは、除外される側を既定値にしている。だからそれを忘れた呼び出し元は、トークンが昔からそうであったように振る舞うトークンを鋳造する。時計は、隣にある文字列のタイムスタンプではなくエポックの列である。文字列の `CURRENT_TIMESTAMP` は方言ごとに書式が違い、それを比較する規則は一方では見えず、他方では間違っているからである。

インスタンスが方針を定め、会社はそれを**厳しく**できるが、緩めることは決してできない。実効値は軸ごとに二つのうち厳しいほうである。これは意図してブランディングの逆である。ブランディングでは会社の文書が丸ごと勝つ — ブランディングは見た目であり、こちらは制御であって、床を所有するのはインスタンスである。強制は `resolve-token` にあり、他のどこにもない。ベアラがプリンシパルになる唯一の場所である。期限の切れたセッションは理由を添えて失効させられる。だからトークンの一覧がなぜ終わったのかを言えるし、期限の切れたセッションを使って蘇らせることもできない。

コンソールは、ポインタの動きではなく最後の**リクエスト**を基準に残り時間を数える。サーバが測るのはリクエストなので、マウスで駆動される時計は、サーバがすでに期限切れにしたセッションを元気に生きているものとして表示してしまう。

14.6　コンソールの方針

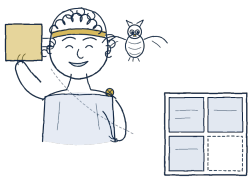
すべての応答は基本となるヘッダの組を運び、コンソールはすべてを同一オリジンに限る Content-Security-Policy を運ぶ。そのスクリプトの出所はいまや `'unsafe-inline'` ではなく**リクエストごとのナンス**である — 前者は、注入されたスクリプトタグを実行させてしまう指定である。

これを手に入れるには、方針の一行より多くの費用がかかった。ナンスはインラインのイベントハンドラを認可できず、コンソールはそれを百四十一個使って作られていた。いまはすべて委譲されたリスナーである。描画のときにクロージャのレジストリへの添字を保持するデータ属性を書き、イベント種別ごとに一つのリスナーがそこへ配る。呼び出しの式は読める場所に残り、その引数はいま、文字列へエスケープして入れて解析し直すのではなく本物の値である — これによって、アポストロフィを含む見出しが自分のボタンを壊しうるという一群の引用符バグも消えた。スタイル属性は許されたままである。二百五十個が残っており、注入された CSS は注入されたスクリプトよりはるかに小さな獲物である。

一つのハンドラがテンプレートリテラルではなく単引用符の文字列の中に座っていたので、そのレジストリ呼び出しは決して補間されず、導線の送信ボタンは死んだ文字列として出荷された。どの静的検査もそれを見なかった。ブラウザのテストがボタンを押し、時間切れになった。いまはインラインのハンドラと、まさにその取り残しの両方に静的検査がある。方針が保たれるのは、それらが保たれているあいだだけだからである。

14.7　開いてみるファイル

domain/authz/secretbox.rkt、domain/authz/authz.rkt（トークン、2FA、回復コード）、cli/telemachus-secrets.rkt、docs/ops/secrets-at-rest-runbook.md、test/secretbox-tests.rkt、test/console-tests.rkt。



Chapter 15

TMach を拡張する

15.1　四つの継ぎ目と、一つの接頭辞

プラグインはマニフェストと入口モジュールを持つディレクトリであり、起動時に読み込まれる。それは四つの継ぎ目を埋めることができ、拡張の契約はその四つで尽きている。

- **ツール** — `(provide tools)`。モデルが呼べる能力である。組み込みと同じレジストリを通して登録されるので、ツールごとの権限とチームごとの有効化が、プラグインが何もしなくても適用される。
- **ワークフロー** — `(provide workflows)`、またはディレクトリに置いた仕様ファイル。API から公開された仕様とまったく同じように検証され、最初に参照された時点でチームへ実体化される。プラグインは読み込みの時点でチームを持たないからである。
- **HTTP ルート** — `(provide routes)`。`/api/x/<plugin-id>/...` に載る。接頭辞はプラットフォームが固定しているので、プラグインが中核のルートや他のプラグインのルートを覆い隠すことはできない。すべてのルートはベアラトークンを要求し、それが名指す権限はハンドラが走る前に検査される。そして形の崩れたエントリは、一時間後のリクエストではなくそのプラグインの読み込みを失敗させる。
- **ジョブ種別** — `init!` から登録し、`x.<plugin-id>.<name>` と名付ける。プラグインの種別は普通のジョブである。投入したチームとユーザー、同時実行の上限、クォータの受け入れ判定、組織のゲート、キャンセル、リースのすべてが適用され、そのどれもプラグインから受け継がれない。遠隔の種別は検証関数を供給しなければならない。代わりに厳しくしてくれるインプロセスのハンドラが存在しないからである。

15.2　プラグインはどこで見つかるのか、そして見つからないとどうなるのか

組み込みのディレクトリは常に探される。`TMACH_PLUGINS` は `:` 区切りの一覧としてそこへ足す。そして文字どおりの `-` というエントリは、自分のものだけを出荷するつもりの配備のために組み込みを外す。これはかつて読み込む唯一のディレクトリを名指していたので、自分のプラグインを足すとドキュメントパイプラインと他の同梱プラグインすべてが静かに消えた — ワークフローが失われたまま健全そうに立ち上がるインスタンスである。

その形 — 健全に見えて静かに欠けている — には名前を付ける価値がある。ローダはそれを二つ目も抱えていたからである。読み込み中に例外を上げたプラグインは捕まえられ、記録

され、飛ばされる。それは正しい。一つの悪いプラグインがサーバを止めてはならない。しかしそれは記録されるだけで、六十行の中の一行だった。そしてインスタンスはその後、そのプラグインのツール、ワークフロー、ルート、ジョブ種別がただ無い API を提供した。いまは失敗が保たれる。プラグインの一覧の隣に `failed` の一覧があり、起動時に幾つなぜ失敗したのかを言う警告の塊があり、そもそも起動を拒否する厳格モードがある。二つのプラグインが一つの ID を主張することも、静かな覆い隠しではなく失敗である。

関連する罠を一つ。まさにこれを生んだからである。`raco make` はプラグインの入口モジュールに届かない。ローダがそれを `dynamic-require` するからである。したがって Racket の更新は `plugins/<id>/compiled/` の下に読み込めない古いバイトコードを残す — そして上記の変更より前は、それを誰も読まない一行で言っていた。

15.3　自分の画面

プラグインは自分のページを二つのディレクトリから配信し、その違いは誰が読んでよいかである。`landing/` は `/beta/bundle/<id>/` で公開され、公開キャッシュに載る — 見込み客にリンクを送るマーケティングの導線である。`bundle/` は `/api/x/<id>/bundle/` で認証を要し、ベアラトークンを要求し、`private, no-store` で応答される。顧客向けの画面は後者に属する。それが存在する前は前者だけがあり、サインイン済みの画面をそこから配信すれば、顧客のページを公開キャッシュの後ろに置くことになっていた。

資産はバンドルの中に入れて出荷する。コンソールの方針は同一オリジンなので、CDN のフォントやスクリプトは遮断される — 意図してそうしてある。

その認証付きディレクトリについて、その上に何かを建てる前に知る価値のあることが二つある。そこのファイルは、チェックアウト配下の固定された経路ではなく、ローダがそのプラグインを見つけた場所から配信される。別の場所から読み込まれたプラグインは、かつてツールとルートを完璧に配信しながら、持っているページはすべて 404 を返した。そしてそこのページは、ブラウザで単に開くことができない。トークンはコンソールの保管庫に住んでおり、ページ遷移はベアラを運ばないからである — スモークが一年通り続けたのは、`curl` がブラウザの決して送らないヘッダを送るからである。開くには、手元にあるトークンで短命な**チケット**を鋳造する。最初の応答がそれを、そのプラグインのバンドル経路に限ったクッキーとして設定するので、ページ自身のスクリプトとスタイルシートは、すべての URL に引きずられることなく後に続く。サインイン時に発行されるセッションクッキーではない。JSON の API はクッキーなし・CSRF の面なしという性質を保ち、こちらは一つのプラグインの静的ファイル以外のどこにも届かない。

15.4　顧客の色をまとう

ブランディングの文書はテーマを運ぶ。背景、表面、文字、控えめな文字、ブランド、ブランドの上に乗る文字、角の丸み、モード、そしてフォントである。これはオンボーディングの導線がすでに使っていたのと同じトークンの語彙なので、配色は一度書けば両方がまとう。そして組織ごとの扱いが無料で付いてくる。会社のブランディングは同じ鍵の下のもう一つの文書だからである — 自分のホスト名を持つ会社は、誰かがサインインする前からテーマをまとう。

二つの規則がこれを正直に保つ。未知のトークンは無視ではなく拒否される。静かに何もしないトークンは、組み込み側がテーマ機能が壊れていると結論する道だからである。そしてコントラストはサーバによって強制され、単に警告されるのではない。本文、控えめな文字、パネル上の文字、ボタンのラベルには WCAG AA、ブランドとその地には低めの基準である。これは人がサインインする画面であり、自分のサインインのリンクを見えない色にした

インスタンスには、UI を通って戻る道がない。ボタンの地はトークンではなくコンソールが混ぜる色なので、検査されるのはその混色である — ブランドそのものを検査すれば、完璧に描かれるテーマを落とし、そうでないものを通してしまう — そして配色のどちら側が暗い側なのかは、モードのトークンから読むのではなく測って決める。

15.5　使えないもの

はっきり述べておく。誰も望みの上に設計しないために。プラグインは中核のコンソールにタブを足すことができない — 代わりに自分の画面を配信する。機能フラグはチームごとであり、インスタンス全体の切り替えはないので、「この配備にはチャットが無い」はすべてのチームについてそうすることを意味する。そしてホットリロードはないので、プラグインを足すには再起動が必要である。

15.6　開いてみるファイル

`docs/integrators-guide.md`。`plugins/integrator-demo/` は手本となる実例である — ツール一つ、ルート二つ、ジョブ種別一つ、そしてテーマを当てた画面。`domain/agent/plugins.rkt`、`domain/branding/branding.rkt`、`docs/reference/sdk.md` と `docs/reference/plugins.md`。



Chapter 16

このプロジェクトはどう試験するのか

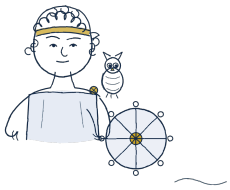
- **ユニットスイート**（`test/*-tests.rkt`）は、新しいデータベースに対して `domain/` を直接動かす。両方の方言で走る。パイプライン全体も、テストのプロセスの中で待ち行列を汲み出すことで、本物のスケジューラを通して走る。
- **スモークスクリプト**（`test/server-smoke.sh` とその仲間）は本物のサーバを起動し、`curl` で HTTP 越しに検査する。モデルだけでなくエンドポイントを試験すること。翻訳マネージャで実際に噛みついたバグは HTTP の層に住んでいた。問い合わせの絞り込みが文字列をシンボルの鍵と比べ、静かに既定値へ落ちていたのである。
- **端から端までのゲート**（`test/e2e/validate.sh`）はヘッドレスのブラウザでコンソールを操り、捕まえられなかったページのエラーと 5xx のいずれでも失敗し、台本を与えた模擬モデルに対してドキュメントパイプライン全体を走らせる。これが配備のゲートである。スクリーンショットの巡回は何も検査しないので、壊れたページでも写真に撮る。
- **モックはサーバである**。`test/mock-llm.rkt` は決定的な OpenAI 互換のモデルで、エージェントループのためのツール呼び出しモードと、パイプラインのためのチャットモードを持つ。
- 何かを完了と呼ぶ前に **PostgreSQL で確かめる**。すべてが `DATABASE_URL` を尊重する。
- **CI のゲート**は、ユニット、スモーク、マルチテナンシー、本物の `aws` クライアントに対する S3 エンドポイント、翻訳のゲート（英語は必須、他のロケールは助言）、ドキュメントの差分ゲート、端から端までのゲート、そして導線のブラウザテストである。

飛ばせるスイートは隠せるスイートである。 S3 のスモークは `aws` の CLI が無いとき綺麗に終了する — だから資格情報の秘密鍵が保存時に暗号化されたとき、署名付きリクエストごとにそれを復号する経路は一度も動かされなかった。どの開発機にも、どの CI の実行環境にも、そのクライアントが無かったからである。いまクライアントは開発シェルに入っており、CI でも導入される。そしてスイートは秘密を封じた状態で走る。それが本物の配備が走る姿である。同じスイートは Nix のシェルの外で走ったことがなかったので、自分のコレクション経路を設定する必要に出会ったこともなかった。CI での最初の実行はライブラリを読み込めず、「サーバが立ち上がらなかった」と報告した。

ブラウザにしか見えないものがある。コンソールのインラインハンドラを変換したとき、ちょうど一つがテンプレートリテラルではなく引用符の文字列の中に残った。だからそれは文字どおりのテキストとして描かれ、導線の送信ボタンは何もしなかった。静的検査はすべて通った。ブラウザのテストがボタンを押し、時間切れになった。静的検査が可能なところで

は、後から書くこと。あの変換が依存する二つの不変条件は、いまコンソールをファイルとして読むユニットテストに固定されている。

テストが教えてくれた習慣をいくつか。同じ秒に作られた二つの行に「新しい順」を決して検査しないこと。背景のサーバは PID で殺し、パターンで殺さないこと（`pkill -f` は自分自身のコマンド行にも合致する）。ドキュメントを作るテストは、ブログのルートを一時ディレクトリへ向けなければ、チェックアウトの中へ書き込んでしまう。



Chapter 17

運用の覚書

- **経路は定義の場所で固定する。**ウェブサーバはリクエストを処理しているあいだ、現在のディレクトリを自分のウェブルートへ向け直す — パッケージとして導入した場合は、読み取り専用の Nix ストアの中である。遅延して解決される相対経路は、そのストアを狙う。ブロブのルートは構造上絶対経路であり、ユニットテストがそれを検査し、サーバは相対経路では起動を拒否する。これは一度出荷されてしまった。起動は健全で、すべてのユニットテストが通り、最初のドキュメント保存が「Permission denied」で失敗した。

- **クォータは待ち行列を止める。**ワークフローのステップはスケジューラのジョブであり、チームが AI 予算の内側にあるときだけ受け入れられる。既定の一日二千トークンでは、一括の下書きや一括の抽出は最初の数ジョブで止まり、停止したように見える。先に `ai.tokens.total` を上げること。自動化のカードが残りの予算を表示しているのは、この理由である。

- **一メガバイトを超えるアップロードにはリスナーの上限を上げる必要がある。**そうでなければ、下にいるウェブサーバが応答を一切返さずに接続を落とす。

- **コンソールは起動時に自分の HTML をキャッシュする。**編集を反映するには再起動すること。

- **マルチテナンシーはフラグを立てるために一度の再起動を要する。**その後に会社を迎え入れるのに再起動は要らない。

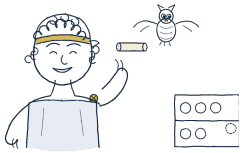
- **PDF のテキスト抽出には poppler-utils が必要である。**抽出器が欠けていればインデックス化の実行は意図して失敗する。どの PDF も何か月も静かに検索不能であることがないようにである。

- **バックアップを取る前に、保存時の秘密について決めること。**鍵を設定していなければ、データベースのダンプはすべての TOTP の種と S3 の秘密鍵を平文で運ぶ — それは支持される選択であり、いまは見える選択である。鍵の採用はコマンド一つで停止時間もない。鍵をデータベースと同じバックアップに入れておいても何も得られない。鍵を失えば、すべての二要素認証を再登録し、すべてのアクセスキーを再発行することになる。

- **サーバを更新すればワーカーも更新することになる。** pull 型のワーカーは、すべての心拍と結果に確保の隔離トークンを返さなければならない。返さないものは 409 を受ける。リファレンスのワーカーはそれを送るが、第三者のワーカーは歩調を合わせて更新する必要がある。

- **テーマは拒否されうる**。コンソールの配色は書き込みの時点で検証され、コントラストのゲートを通る。だから読めない組み合わせを求めた運用者は、サインインの画面で気づくのではなく、どの対がどれだけ足りないのかを告げられる。

`docs/ops/` の下のランブックが、ドキュメント保管庫、ワークフローエンジン、マルチテナンシーを運用者の言葉で扱っている。



Chapter 18

貢献する

- 始める前に `CLAUDE.md` を読むこと。そして時間を費やして学んだことがあれば、そこへ書き足すこと。それがこのプロジェクトの記憶である。
- SQL は方言に依存させず、スイートを両方の方言で走らせること。
- モデルに面するすべての機能は検証し、拒否する。すべての AI 呼び出しは計量される。すべてのリソースは `can?` を通して検査される。
- 静かに劣化するのではなく、声を上げて拒否すること。読めない返答、未知のテーマのトークン、復号できない秘密、モデルが守らなかったスキーマ — これらはすべて自分の名を告げるエラーであり、空文字列でも、落とされたフィールドでも、静かな既定値でもない。このプロジェクトが追いかけることになったバグのほとんどは、何かが礼儀正しく失敗していたものだった。
- ルート、ツール、ワークフロー、権限を足せば、生成されるリファレンスが変わる。同じ変更の中で再生成してコミットすること。
- 利用者に見える新しい文字列は、三つのうち正しい住所へ置き、それから抽出器を通す。ゲートが必須とするのは英語である。他のロケールは網羅率である。
- 方針の分岐があるものは、まず設計すること。`docs/design/` の下に短い文書を置き、データの形、どのバックエンドでも実装できる契約、そして確認すべき決定を書く。決定ログは `docs/design/decisions.md` である。
- コミットメッセージは、何をどう検証したのかを言う。スライスが完了するのは、ユニットスイート、スモーク、ゲート、そして `nix build` が、両方のデータベースで一致したときである。